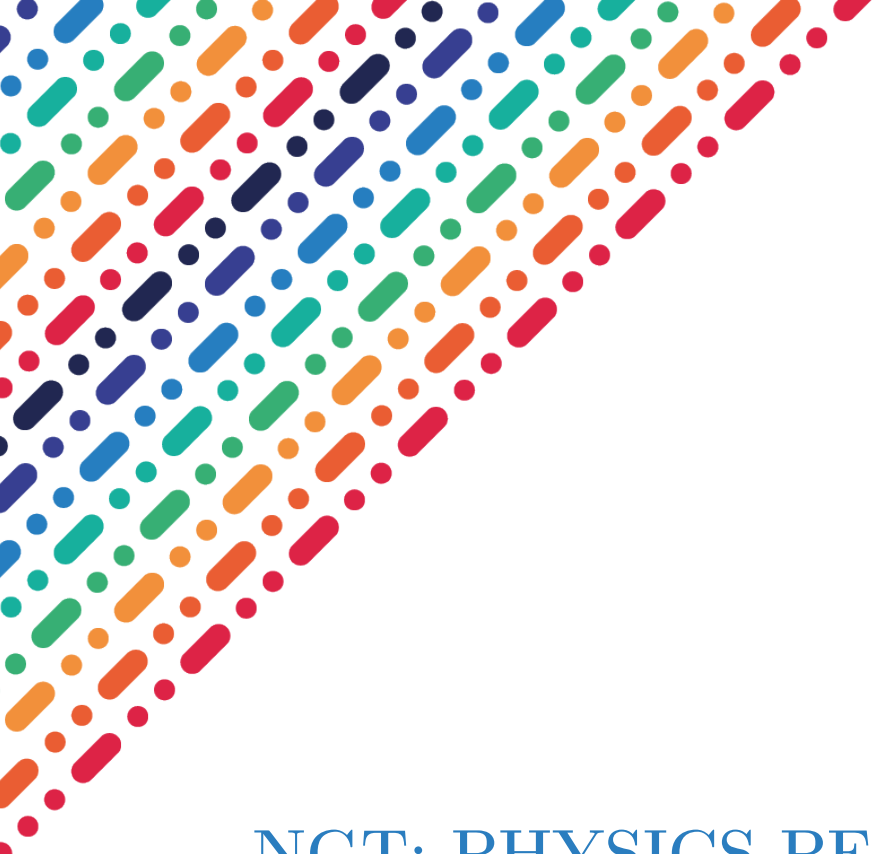




NGT: PHYSICS PERFORMANCE EVALUATION OF CONTINUOUS REAL-TIME CALIBRATIONS WITHIN THE CMS HIGH-LEVEL TRIGGER SYSTEM

August 2026

AUTHOR(S):

Maximilian Spannring

SUPERVISOR(S):

Jessica Prendi (CMS Collaboration)

Marco Musich (CMS Collaboration)





PROJECT SPECIFICATION

Accurate physics reconstruction of collision events at the LHC relies on precise knowledge of detector conditions at the moment an event is acquired. As these conditions fluctuate during data-taking, calibrations must be derived to account for them. Within the Next Generation Triggers (NGT) project, a novel approach is being tested to derive calibrations for online reconstruction at the High-Level Trigger (HLT) with much higher frequency than current operations. A demonstrator has been commissioned to test the feasibility of this new paradigm for the Phase-2 conditions framework, with the aim of deriving calibrations continuously within a strict 8-hour window while using limited computing resources and maintaining physics quality. This summer student project will focus on a retrospective, comprehensive physics validation of this demonstrator using results collected during the final Run 3 data-taking period. We will perform a comparative analysis between the conditions derived by the NGT Demonstrator, the standard HLT conditions, and the high-precision Offline conditions which serve as the reference truth. The goal is to quantify improvements in physics object reconstruction and assess triggering efficiencies, to determine if the continuous calibration approach is feasible for Phase-2.



ABSTRACT

Within the Next Generation Triggers (NGT) project, Task 3.4 (SAKURA) investigates deriving calibrations for the CMS High-Level Trigger (HLT) far more frequently than in current operations, with the aim of bringing offline-quality calibrations online. A demonstrator implementing this continuous calibration loop was deployed in the CMS DAQ and HLT systems towards the end of LHC Run 3. This report presents a retrospective physics validation of the demonstrator on 2026 collision data. For this purpose, an evaluation pipeline was developed that automates run selection via the CMS Online Monitoring System, extraction of run-specific HLT configurations, large-scale re-reconstruction on HTCondor with three sets of conditions – the standard HLT conditions, the demonstrator (NGT) conditions, and the offline prompt conditions serving as reference – and the production and comparison of data-quality histograms. Overall, the pipeline enables a faster and more streamlined data analysis. Applied to 14 runs of the Run2026D EGamma datasets, the evaluation shows that the invariant mass distribution of $Z \rightarrow e^+e^-$ candidates reconstructed with the NGT conditions closely follows the prompt reference in both peak position and resolution.



TABLE OF CONTENTS



1	INTRODUCTION	4
2	Background	5
2.1	Calibration Workflows at CMS	5
2.1.1	Calibrations and Conditions	5
2.1.2	Data Streams and the Prompt Calibration Loop	6
2.2	The NGT Demonstrator	6
3	The NGT Evaluation Pipeline	8
3.1	CMS Online Monitoring System	9
3.2	CMS Data Aggregation System	11
3.3	Extracting the HLT Configuration	11
3.4	HTCondor Batch Submission	12
3.5	Data Quality Monitoring	13
3.6	Plotting	14
4	Results	14
5	Conclusion	14
6	REFERENCES	18

1 INTRODUCTION

The upcoming High-Luminosity upgrade of the Large Hadron Collider (HL-LHC) will increase the instantaneous luminosity by almost an order of magnitude, from $\mathcal{L} = 10^{34} \text{ cm}^{-2} \text{ s}^{-1}$ to $\mathcal{L} = 7.5 \times 10^{34} \text{ cm}^{-2} \text{ s}^{-1}$ [3]. This is why the already highly sophisticated data acquisition (DAQ) system of the Compact Muon Solenoid (CMS) detector has to be upgraded even further. The trigger system is a crucial part of the DAQ: it reduces the raw detector readout to a manageable data volume. It consists of two stages, the hardware-based Level-1 Trigger (L1T) and the software-based High-Level Trigger (HLT). Once an event is accepted by the trigger system, a first full reconstruction pass – the *prompt reconstruction* – takes place at the Worldwide LHC Computing Grid (WLCG) Tier-0 site at CERN. As shown in Figure 1, the raw collision data are filtered from 40 MHz down to 10 kHz before being written to the tape storage system.

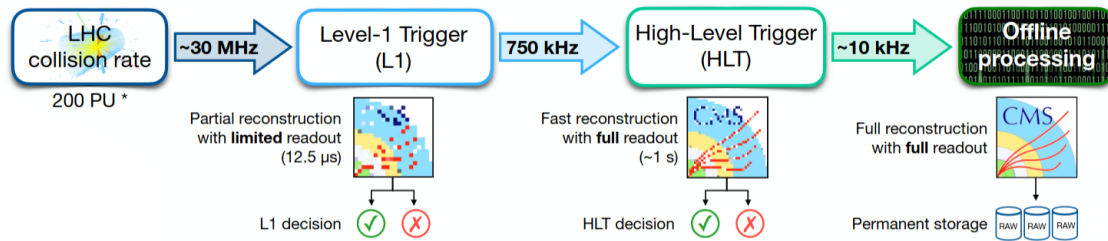


Figure 1: Data flow through the CMS trigger and reconstruction chain. From [14]

However, the crucial question is: what if new, interesting physics is hidden in the bulk of the data that is filtered out? This is the motivation behind the Next Generation Triggers (NGT) project, which aims to enhance the physics reach of both the CMS and the ATLAS experiments beyond their current baselines. The goal is to achieve full event reconstruction at a larger event rate, in real time. Traditional CMS reconstruction is performed offline, takes tens of seconds per event, and is based on algorithms developed decades ago [12].

Work Package 3 of the NGT project rethinks the real-time data processing of CMS, and the Real-time Reconstruction Revolution (R^3) within it works on reconstructing events in high quality online, directly at the HLT. The Speedy Alignment and Calibration Upgrade for Real-time Algorithms (SAKURA) is specifically responsible for optimising the calibration workflows on which that reconstruction depends. It corresponds to Task 3.4 of the NGT programme and is the project this report concerns.

Since the beginning of the SAKURA project, the structure of the CMS calibration workflows has been reviewed and candidate workflows suitable for inclusion in R^3 have been identified [3]. Building on this, a demonstrator was built and deployed within the CMS DAQ and HLT systems towards the end of LHC Run 3; its architecture is discussed in Chapter 2. Some aspects of the demonstrator were evaluated on the 2025 data, however, a thorough assessment of the 2026 data was still missing.

The purpose of this internship was to build a streamlined analysis pipeline and to use it to evaluate the performance of the demonstrator for the 2026 run.

The architecture of this pipeline is described in Chapter 3, and the results of the evaluation in Chapter 4.



2 Background

2.1 Calibration Workflows at CMS

2.1.1 Calibrations and Conditions

Before diving deeper into the derivation of *optimal* calibrations, we have to clarify how calibrations relate to the more practical term *conditions*. Conditions are non-event data resulting from calibration and alignment algorithms [8]. They change over time, require frequent look-ups, and have to be available for offline event processing.

The data model behind conditions is organised as follows (see also Figure 2) [8]:

- A **Payload** contains the parameters related to the different subsystems, such as detectors, triggers, beam monitoring and physics algorithms.
- The **Interval of Validity** (IOV) is a time interval that specifies the validity of a Payload.
- A **Tag** identifies a set of contiguous, non-overlapping IOVs with their associated Payloads.
- A **Record** is a special C++ class that acts as the entry point for a Payload in CMSSW.
- A **Global Tag** is a collective label that links a set of Tags to the Records involved in a given workflow.

The CMS Conditions Database (CondDB) is the specialised database system used to store and manage these conditions [8].

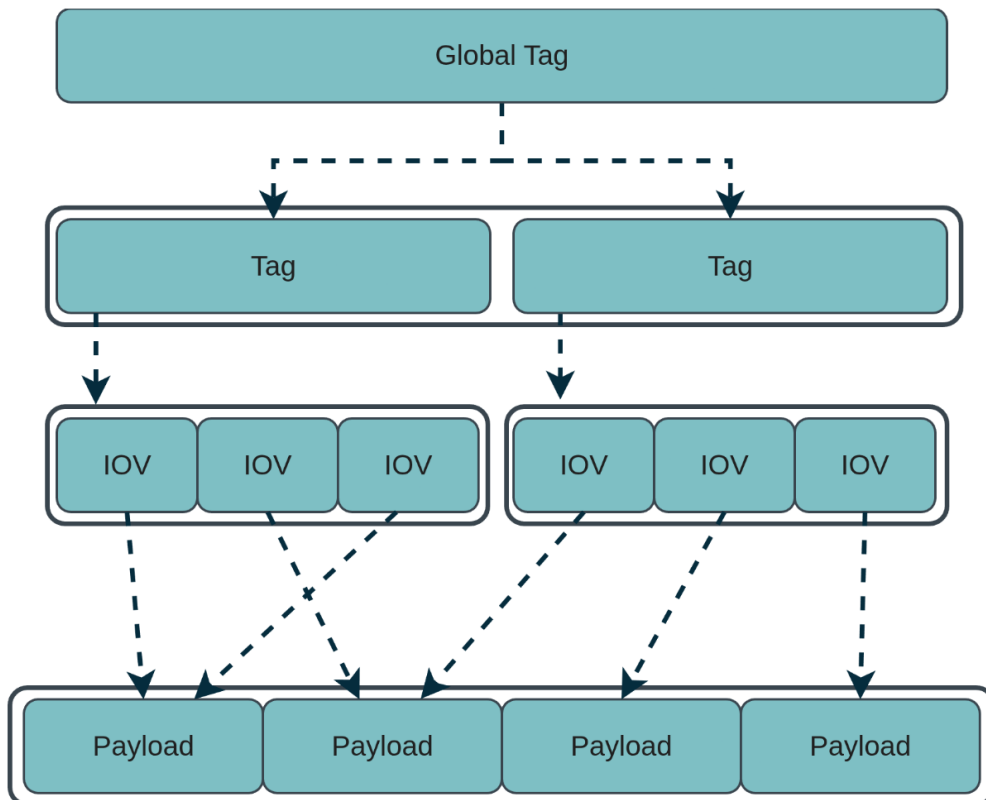


Figure 2: The CMS conditions data model. From [7]



2.1.2 Data Streams and the Prompt Calibration Loop

The HLT splits the accepted data into four categories of primary streams: the Physics, the Calibration, Scouting and the Express stream. These streams are the input to the offline processing on the Tier-0 farm at CERN [5]. They are organised as follows:

- The **Physics stream** is the main data stream for physics analysis. Its full reconstruction pass, the *prompt reconstruction*, is delayed by 48 hours. This delay allows the fast-changing conditions to be computed first. The output of this stream is divided into Primary Datasets.
- The **Calibration stream** consists of events selected at the HLT for calibration purposes and processed at Tier-0.
- The **Express stream** is used for the reconstruction of a limited amount of data in order to give fast feedback on the physics performance and the detector status. Its results are usually available within a few hours.
- The **Scouting Stream**. Scouting means storing a (much) larger set of events for analysis but dropping the original RAW data and storing only a small summary of the processed output from the software trigger [2].

A subset of the data from the Calibration and Express streams is used to compute the updated conditions for a given run, while the bulk of the data is buffered on disk. This workflow is called the Prompt Calibration Loop (PCL) and is illustrated in Figure 3. The PCL has to complete within the 48-hour window, so that the updated conditions are available in CondDB before the prompt reconstruction of the Physics stream begins.

2.2 The NGT Demonstrator

Figure 4 shows a schematic view of the Next-generation REal time Demonstrator (NERD). There are two main data-processing legs: the NGT DAQ and the NGT Calibration Loop. The demonstrator is fed with $\mathcal{O}(1\%)$ of the L1T output, corresponding to $\approx 1.3\text{ GB/s}$, via two dedicated streams (RAW: `LocalTestDataRaw`, scouting: `LocalTestDataScouting`). The data is buffered for 8 hours and then converted into the FED raw data (FRD) format expected by the HLT [3].

The CMS DAQ system, whose hardware the demonstrator reuses, is organised as follows:

- **Readout Units** (RUs) read and merge event fragments into super-fragments.
- **Builder Units** (BUs) assemble the super-fragments into complete events.
- **Filter Units** (FUs), where the HLT application runs and selects the subset of events to be saved for permanent storage.

The SM, BU, FUs and DQM make up the DAQ leg, while the calibration leg consists of the dedicated calibration nodes and the conditions database.

The calibration loop of the demonstrator derives its calibrations in three steps: (1) event reconstruction, (2) calibration algorithms, and (3) uploading of the conditions. Each of these steps is orchestrated by a finite state machine, meaning that the system occupies exactly one state at a time and moves between states via defined transitions.

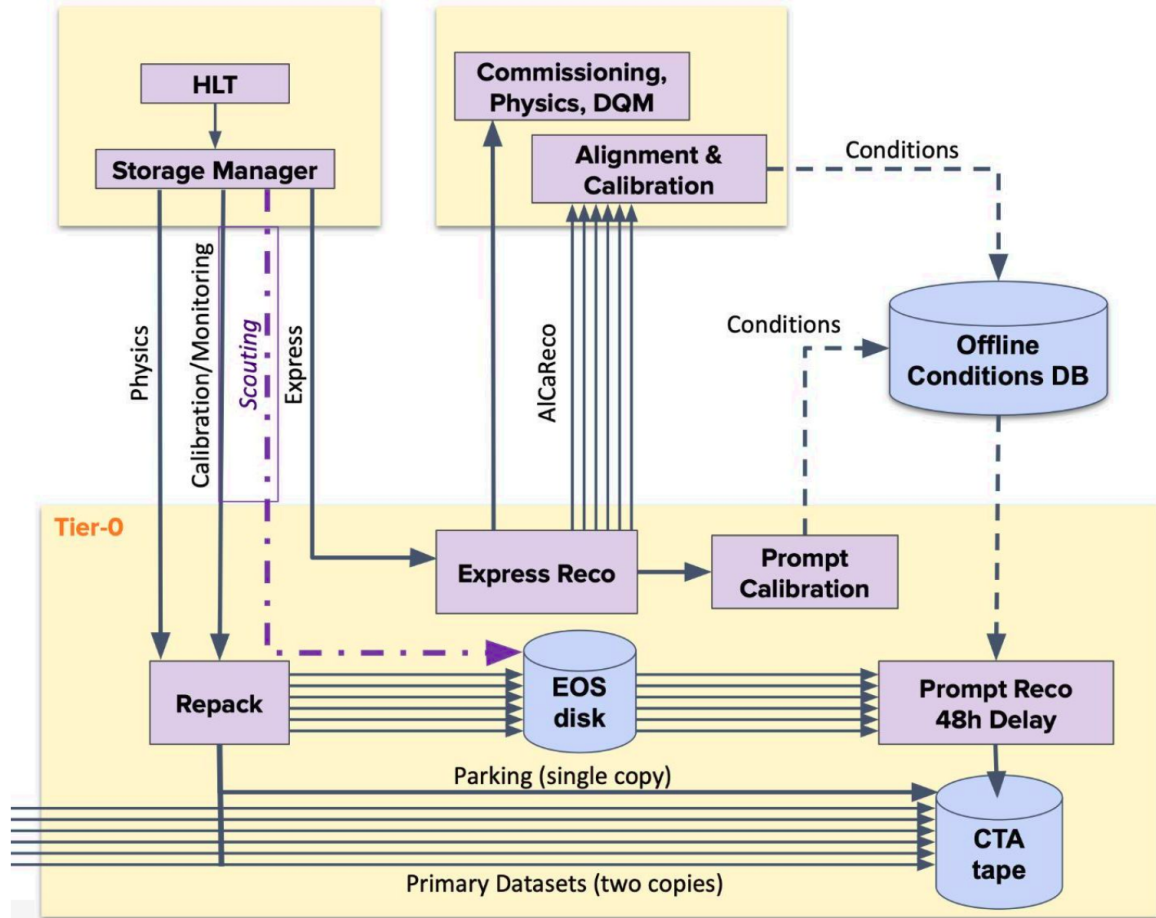


Figure 3: Schematic overview of the Prompt Calibration Loop. From [3]

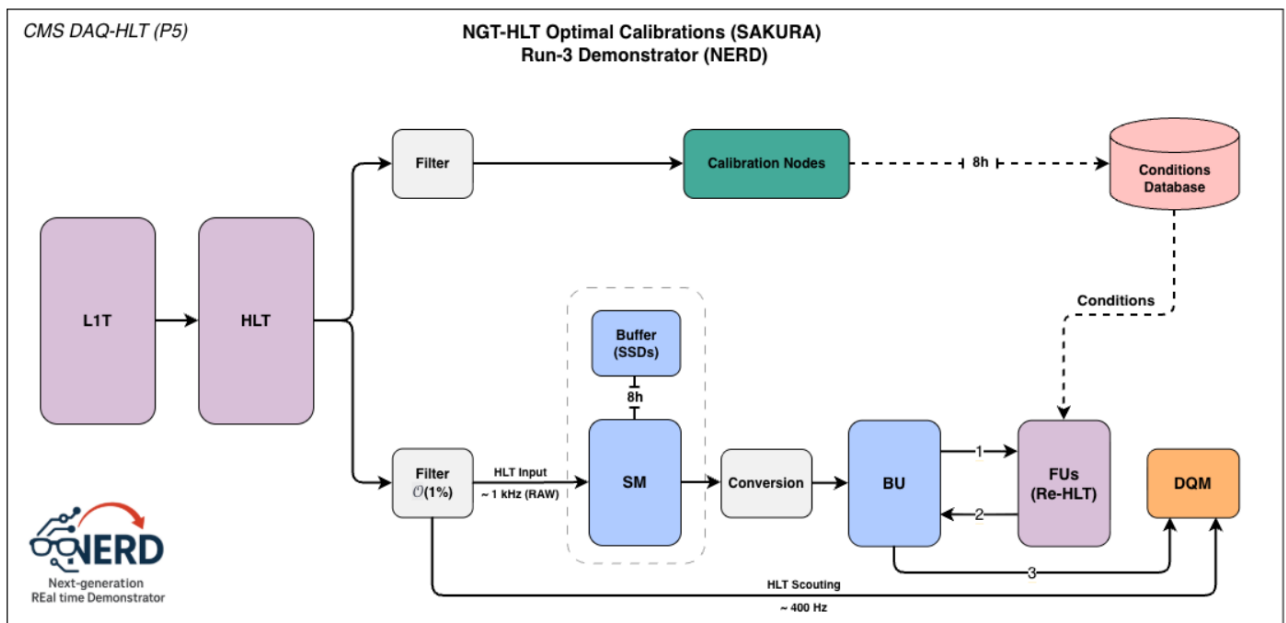


Figure 4: Schematic view of the Next-generation REal time Demonstrator (NERD), showing the two data-processing legs. Adapted from [3].



But which calibrations are actually used in the NGT demonstrator? The choice comes from a broader list resulting from an internal survey on the conditions deemed critical for data-taking. That list was then narrowed down by criticality, operational feasibility and expected physics impact [3]. Ultimately, the three calibrations used in the 2025 run were:

- **ECAL laser transparency corrections**, as they have a strong impact on electron and photon reconstruction quality and add minimal cost, since they are already produced for the prompt reconstruction within the ECAL laser monitoring system.
- **ECAL pedestals**, which also have a large impact on electron and photon performance. These calibrations describe the baseline output level of a readout channel – what the ADC reads when no real signal is present. This matters because the baseline is not identical for every channel and can drift over time, due to electronic noise or thermal changes, for example.
- **Silicon strip tracker bad components**. Over time, some components stop giving reliable readouts due to radiation damage, hardware failure or other problems, which is why bad components in the silicon strip tracker are masked.

For the 2025 run, the demonstrator used a dedicated Global Tag, `150X_dataRun3_NGT_v2`, which is a copy of the production HLT Global Tag `150X_dataRun3_HLT_v1` with the Tags for these three Records replaced.

The task of the Data Quality Monitoring (DQM) system is to assess the quality of the recorded data. It can be used as a *monitoring* tool, raising alarms when parts of the detector malfunction, or as a *data certification* tool, recording for which fractions of the taken data parts of the detector were faulty. Alternatively, it can also generate a large number of histograms, called *monitor elements*, usually stored in the ROOT file format, which detector experts then use to understand what is happening inside the detector [1].

For the demonstrator, new DQM modules for HLT scouting were implemented. Scouting is a data-taking strategy in which a much larger fraction of events - up to the full L1T output - is kept, by storing only compact HLT-level reconstructed objects instead of the full RAW event content [6]. In essence, it trades reconstruction quality for better statistics.

3 The NGT Evaluation Pipeline

For its fast online reconstruction, the HLT cannot derive its own calibrations. The conditions it uses are mostly manual uploads from detector experts or aggregates of PCL-derived conditions over larger amounts of data. Since these are not up to date with the current state of the detector, the online reconstruction is less accurate than it could be. The goal of the NGT demonstrator is to close this gap and bring offline-like reconstruction quality directly to the HLT.

My task for this internship was to build an offline evaluation pipeline and to assess whether the performance of the conditions derived by the NGT demonstrator is noticeably better than the standard reconstruction with HLT conditions and comparable to the prompt conditions. This means executing the HLT reconstruction over CMS raw data three times, with three different sets of conditions, and comparing the results. In the following subsections, I describe the design of the evaluation pipeline, shown in Figure 5, in more detail. The results of the evaluation can be found in Chapter 4.



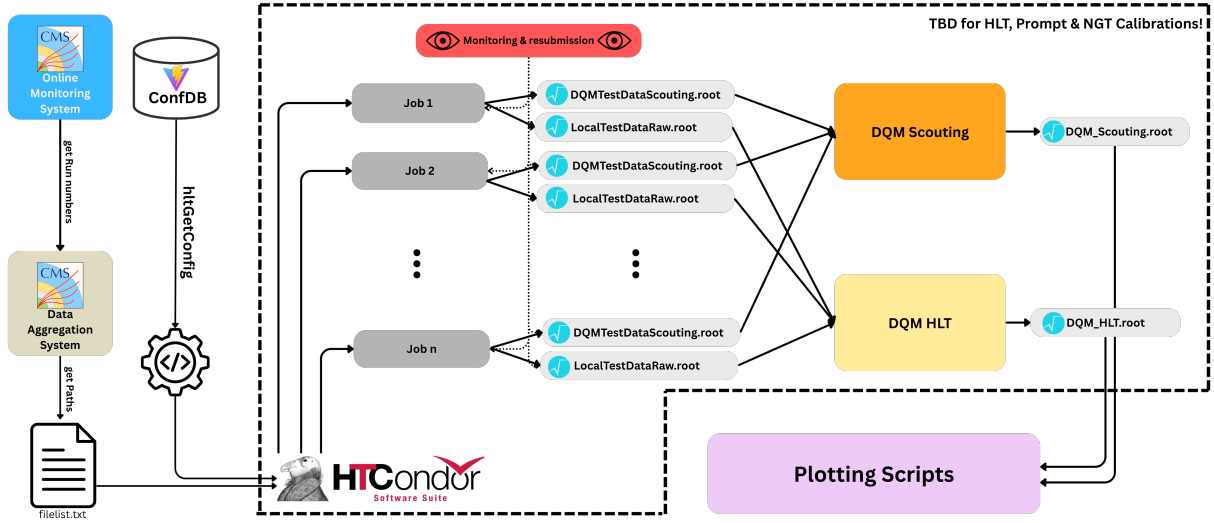


Figure 5: Schematic overview of the NGT evaluation pipeline.

3.1 CMS Online Monitoring System

The first question we face when evaluating the performance of the demonstrator is: which data should we use? Every time CMS starts recording data, that recording session is called a *run* and is assigned a unique run number. Independently of that, the LHC injects and stores the beam in a *fill*, which lasts from hours to a day, and within a single fill multiple runs can occur. A run is further divided into *lumisections* (LS), each corresponding to about 23 seconds. The hierarchy is therefore $LS \subset run \subset fill$.

Each fill is linked to an injection scheme that specifies the pattern in which hadrons are injected into the LHC. This pattern defines how many particles collide at which interaction point and how large the gaps between the bunches are, which in turn influences the electron-cloud build-up and the beam quality. Moreover, not all runs are proton–proton collision runs: there are also cosmic-ray runs, calibration runs, heavy-ion runs, and others.

Consequently, we have to choose runs with the right parameters. For this we use the CMS Online Monitoring System (OMS). The OMS aggregates and integrates different non-event data sources into a central Oracle database, allowing users to view, compare and correlate real-time and historical information. Its user interface is a browser-based platform that provides access to a wide range of data categories [1]. We select runs from the database that match the following criteria:

- collision type has to be proton–proton,
- `stable_beam = true`,
- `l1_hlt_mode = "collisions2026"`.

The CMS OMS API is used to query the online monitoring system.¹ The code is available

¹<https://gitlab.cern.ch/cmsoms/oms-api-client>



in a dedicated Jupyter notebook.² This results in a list of 225 runs in total (37/76/65/47 across the eras 2026A–D). The head of the resulting CSV file is shown in Listing 1.

```
run_number,injection_scheme,run_era,start_time,end_time,snapshot_time
401623,Single_4b_2_2_2_noLRs,Run2026A,2026-03-07T14:56:40Z,2026-03-07T16:03:03Z,2026-03-07
↪ 22:56:40.000
401624,Single_4b_2_2_2_noLRs,Run2026A,2026-03-07T16:04:48Z,2026-03-07T18:13:27Z,2026-03-08
↪ 00:04:48.000
401625,Single_4b_2_2_2_noLRs,Run2026A,2026-03-07T18:16:06Z,2026-03-07T19:03:46Z,2026-03-08
↪ 02:16:06.000
401626,Single_4b_2_2_2_noLRs,Run2026A,2026-03-07T19:07:01Z,2026-03-07T19:55:53Z,2026-03-08
↪ 03:07:01.000
401627,Single_4b_2_2_2_noLRs,Run2026A,2026-03-07T19:57:40Z,2026-03-07T21:29:19Z,2026-03-08
↪ 03:57:40.000
401630,Single_12b_8_8_8_1bpi_12inj,Run2026A,2026-03-07T22:56:21Z,2026-03-07T23:28:29Z,2026-03-08
↪ 06:56:21.000
401631,Single_12b_8_8_8_1bpi_12inj,Run2026A,2026-03-07T23:31:10Z,2026-03-07T23:44:18Z,2026-03-08
↪ 07:31:10.000
401632,Single_12b_8_8_8_1bpi_12inj,Run2026A,2026-03-07T23:46:58Z,2026-03-07T23:58:54Z,2026-03-08
↪ 07:46:58.000
401633,Single_12b_8_8_8_1bpi_12inj,Run2026A,2026-03-08T00:01:38Z,2026-03-08T00:14:23Z,2026-03-08
↪ 08:01:38.000
```

Listing 1: Head of `oms_runs.csv`.

As can be seen, we also store the start and end times of each run, plus the snapshot time 8 hours after the start of the run. This becomes relevant when we obtain the configurations for each run in Section 3.3.

We narrow the list down by selecting only runs from the Run2026D era, as these all share the same injection scheme, `25ns_2460b_2448_2092_2239_144bpi_20inj`. This scheme encodes the following information [9]:

- `25ns`: the bunch spacing (the LHC nominal spacing),
- `2460b`: the total number of bunches per beam,
- `2448`: the expected number of colliding bunch pairs at IP1/IP5, i.e. ATLAS and CMS,
- `2092`: the expected number of colliding bunch pairs at IP2, i.e. ALICE,
- `2239`: the expected number of colliding bunch pairs at IP8, i.e. LHCb,
- `144bpi`: bunches per injection,
- `20inj`: the number of injections used to fill the beam.

The list had to be restricted further still. For the `160X_dataRun3_NGT_v6` Global Tag, not all records are available for all lumisections. The binding constraint was the beam spot record, whose earliest valid IOV starts at run 403747; runs before this fail with a fatal `NoRecord` exception. This is not surprising because the beam spot record was introduced around that time, and such gaps are expected shortly after its introduction. This leaves 14 valid runs (403747, 403748, 403751, 403774, 403775, 403811, 403818, 403863, 403864, 403866, 403867, 403894, 403895, 403937), spanning all six EGamma stream instances (EGamma0–5), for a total of 2868 input RAW-RECO files.

²https://github.com/maxspannring/CERN_Openlab_internship2026/blob/main/pipeline/OMS_API_query.ipynb



3.2 CMS Data Aggregation System

The CMS Data Aggregation System (DAS) is used to track collision data (as well as Monte Carlo samples) distributed across the different sites of the Worldwide LHC Computing Grid (WLCG). We use the DAS Go client to find the file paths for our runs with a dedicated script.³ The output files are saved in a dedicated directory. The code snippet in Listing 2 concatenates the output into a single file list.

```
FILELIST_DIR="oms-api-client/filelists"
MIN_RUN=403747
MAX_RUN=403937

for f in "$FILELIST_DIR"/run_*.txt; do
    run=$(basename "$f" .txt | sed 's/run_//')
    if [[ "$run" =~ ^[0-9]+$ ]] && [ "$run" -ge "$MIN_RUN" ] \
        && [ "$run" -le "$MAX_RUN" ]; then
        cat "$f" >> filelist_NGT_combined.txt
    fi
done
```

Listing 2: Concatenating the per-run file lists into a single list.

The result is a file list of the following form:

```
/store/data/Run2026D/EGamma0/RAW-RECO/ZElectron-PromptReco-v1/000/403/747/
↪ 00000/56af2b70-716f-4ff1-a1a9-2a2152f80a68.root
/store/data/Run2026D/EGamma0/RAW-RECO/ZElectron-PromptReco-v1/000/403/747/
↪ 00000/b78ea30f-66ce-4ba1-9c0a-fd5f02605b7b.root
/store/data/Run2026D/EGamma0/RAW-RECO/ZElectron-PromptReco-v1/000/403/747/
↪ 00000/c0614ed1-9a15-45e4-93ec-80500ffc29e4.root
/store/data/Run2026D/EGamma0/RAW-RECO/ZElectron-PromptReco-v1/000/403/747/
↪ 00000/ccd4536a-6541-483e-aa75-96ca413dfcef.root
/store/data/Run2026D/EGamma0/RAW-RECO/ZElectron-PromptReco-v1/000/403/747/
↪ 00000/ebba4f07-5c7e-4f7a-a982-a575c30b2fe3.root
```

Listing 3: Excerpt of the combined file list.

Overall, EGamma datasets are chosen as the primary datasets because we expect the biggest improvement of the NGT demonstrator on electron and photon reconstruction.

3.3 Extracting the HLT Configuration

All CMSSW code is run by passing a configuration file (`_cfg.py`) to the CMSSW executable, `cmsRun` [15]. The next step in the pipeline is therefore the `01_make_config.sh`⁴ script, which uses the `hltGetConfiguration` command to extract a configuration for each run. In this subsection I explain how it works and why it is necessary.

³https://github.com/maxspannring/CERN_Openlab_internship2026/blob/main/pipeline/generate_filelists.sh

⁴https://github.com/maxspannring/CERN_Openlab_internship2026/blob/main/pipeline/01_make_config.sh



The `hltGetConfiguration` command is a wrapper around the bare-bones `edmConfigFromDB` script that knows how to adapt an HLT menu, normally intended for running online, to different purposes [11]. It returns the configuration of a ConfDB menu, adding customisations for offline testing (I/O with ROOT files) plus a number of features for users, such as re-emulation of the L1T, changing the Global Tag, selecting a subset of paths, and appending user-defined customisations. The output is a large Python file, on the order of 10^5 lines, that materialises one specific trigger menu, containing the different HLT paths. Each path is defined by a sequence of reconstruction (`EDProducer`) and filtering (`EDFilter`) modules [10].

In the NGT evaluation pipeline, the individual parameters for the command are stored in a `pipeline.cfg` file. In essence, the command is run with the parameters shown in Listing 4.

```
# globaltag can also be "160X_dataRun3_Prompt_v1" or "160X_dataRun3_NGT_v6"
hltGetConfiguration "/dev/CMSW_16_0_0/GRun" \
  --globaltag "160X_dataRun3_HLT_v1" \
  --data \
  --unprescale \
  --output none \
  --max-events -1 \
  --eras "Run3_2026" \
  --l1-emulator uGT --l1 "L1Menu_Collisions2026_v1_1_0_xml" \
  --paths "Dataset_TestDataRaw, LocalTestDataRawOutput, \
          Dataset_TestDataScouting, DQMTestDataScoutingOutput, \
          DST_PFScouting*, HLT_TestData_v*, HLTriggerFinalPath, \
          HLT_Ele23_Ele12_CaloIdL_TrackIdL_IsoVL_DZ_v*" \
  --input " " \
  > "hltData_${tag}_${run}.py"
```

Listing 4: Invocation of `hltGetConfiguration`. The Global Tag is the parameter varied between the three reconstruction passes.

Two caveats are worth noting. First, in order to select the correct configuration for the NGT Global Tag, it is essential to specify a snapshot time. Since one IoV is allowed to have multiple payloads, specifying a snapshot time guarantees that the conditions younger than the running time are discarded. Second, in order for the configuration to be processable by the HTCondor batch submission system (the next step), `edmConfigDump` has to be run to create a “dump” of the configuration. Both are handled by the script.

3.4 HTCondor Batch Submission

As a next step, we have to emulate the triggering process at the HLT, in order to see which events are selected and reconstructed differently depending on the conditions used. Because this process is compute-heavy, we use the HTCondor batch submission system to complete it in a reasonable amount of time. HTCondor is an open-source high-throughput computing framework for coarse-grained distributed parallelisation of computationally intensive tasks [13].

To use HTCondor we have to write a submission script that specifies which executable should be run, where it can find the credentials for the grid, and where to write the output, error and log files of the job. It also declares the resources the job requests, and sets a wall-clock budget – in our case `+JobFlavour = "workday"`, which terminates a job after 8 hours. A typical submission file used in the pipeline, here for the HLT tag, is shown in Listing 5.



```
executable = $(jobscrip)
use_x509userproxy = true
x509userproxy = x509up_u<uid>
output = $Fp(jobscrip)hlt.stdout
error = $Fp(jobscrip)hlt.stderr
log = $Fp(jobscrip)hlt.log
request_memory = 5000
+JobFlavour = "workday"
queue jobscrip from <pipeline_dir>/jobs_to_run_HLT.txt
```

Listing 5: Example HTCondor submission file, `condor_HLT.sub`.

The executable that the jobs call is `cmsRun`, with the configurations from the previous step as its argument. This generates two output ROOT files per job.

In the NGT evaluation pipeline, the generation of the HTCondor submission scripts is orchestrated by the `02_submit.py`⁵ script. The submission scripts it generates are then submitted manually with `condor_submit condor_{TAG}.sub`. In addition, the script writes the expected output to a `manifest_{tag}.tsv` file, so that it can later be checked whether the jobs really wrote their output to the filesystem rather than merely exiting with status 0. For a large submission consisting of a few hundred jobs, some are expected to fail; for this reason I wrote the `03_check.py`⁶ script, which gives an overview of the status of the jobs and creates a resubmission file for those that failed.

3.5 Data Quality Monitoring

Once the output files have been successfully written to the EOS filesystem for every job, we execute the DQM HLT client and the DQM Scouting client to produce the monitor elements from those files. We are essentially running these commands from listing 6.

```
cmsRun $CMS_SW_SRC/DQM/Integration/python/clients/hlt_dqm_sourceclient-live_cfg.py \
inputFiles=root://eosuser.cern.ch/eos/.../HLT_run<R>_job0_LocalTestDataRaw.root,...

cmsRun $CMS_SW_SRC/DQM/Integration/python/clients/scouting_dqm_sourceclient-live_cfg.py \
inputFiles=root://eosuser.cern.ch/eos/.../NGT_run<R>_job0_DQMTestDataScouting.root,...
```

Listing 6: Invocation of the DQM HLT and DQM Scouting clients.

Everything is wrapped in the `04_run_dqm.sh`⁷ script, which provides some additional control logic. This stage is not well suited to the HTCondor batch submission system, so it is run on `lxplus` instead. The output ROOT files are written to a dedicated upload directory; the script handles the logic that prevents them from overwriting each other and moves them to a directory specified in `pipeline.cfg`.

To parallelise the process, it was run on six different `lxplus` nodes, each handling a different stream/tag combination.

⁵https://github.com/maxspannring/CERN_Openlab_internship2026/blob/main/pipeline/02_submit.py

⁶https://github.com/maxspannring/CERN_Openlab_internship2026/blob/main/pipeline/03_check.py

⁷https://github.com/maxspannring/CERN_Openlab_internship2026/blob/main/pipeline/04_run_dqm.sh



The DQM stage produces a large volume of log files, amounting to several gigabytes, which is tedious to impossible to analyse by hand. For this reason I wrote two additional scripts,^{8,9} `summarize_logs.py` and `compare_logs.py`. The former reduces the log files to a compact summary, while the latter reads those summaries and diffs them across the three calibration tags. This makes it much faster to find anomalies in the DQM process.

3.6 Plotting

The DQM stage leaves us with $2 \times 3 \times 14$ output ROOT files, one for each combination of DQM client, Global Tag and run. As a final step, the histograms have to be aggregated over each run and compared between the three sets of conditions. This is done by the plotting scripts,¹⁰

My contribution was to refactor these scripts to remove the redundancy between them. The shared logic now lives in the `scouting_plot.py` module, which reads a `config.yaml` holding the settings that apply to all plots (conditions, luminosity, year) and defines an abstract base class serving as a blueprint for the plotting process. Each plotting script is a thin entry script that subclasses this blueprint and overrides its hook methods for the plot-specific details: histogram names, axis labels and axis limits. Settings that affect every plot are therefore changed once in `config.yaml`, while anything specific to a single plot is changed in that plot's subclass.

4 Results

This chapter presents the physics results obtained with the NGT evaluation pipeline described in Chapter 3. They show that the optimal calibration architecture yields a measurable improvement in reconstruction performance, particularly for e/γ objects [4].

Figure 6 shows the invariant mass distribution of the dielectron system for the $Z \rightarrow e^+e^-$ process. The distribution obtained with the prompt conditions, which is expected to be the most accurate, peaks close to the mass of the Z boson, while the distribution obtained with the standard HLT conditions is shifted towards lower mass values. The NGT conditions follow the prompt conditions closely. The mean μ and the standard deviation σ are obtained by fitting the invariant mass distribution in the range 60–120 GeV with a double-sided Crystal Ball function combined with an exponential background. The NGT conditions also yield a better mass resolution than the standard HLT conditions [4].

Figure 7 shows the per-fill stability of the fitted mean mass μ (top panel) and of the relative mass resolution σ/μ (bottom panel) for the $Z \rightarrow e^+e^-$ invariant mass distribution, as a function of integrated luminosity. The NGT measurements lie close to those obtained with the prompt conditions, for both the fitted mean μ and the relative resolution σ/μ [4]. For

5 Conclusion

During this internship, an evaluation pipeline for the NGT demonstrator was built, automating run selection via OMS, the extraction of run-specific HLT configurations, large-scale reconstruction on HTCondor under three sets of conditions, and the production and compari-

⁸https://github.com/maxspannring/CERN_Openlab_internship2026/blob/main/pipeline/summarize_logs.py

⁹https://github.com/maxspannring/CERN_Openlab_internship2026/blob/main/pipeline/compare_logs.py

¹⁰https://github.com/maxspannring/CERN_Openlab_internship2026/tree/main/pipeline/plotting

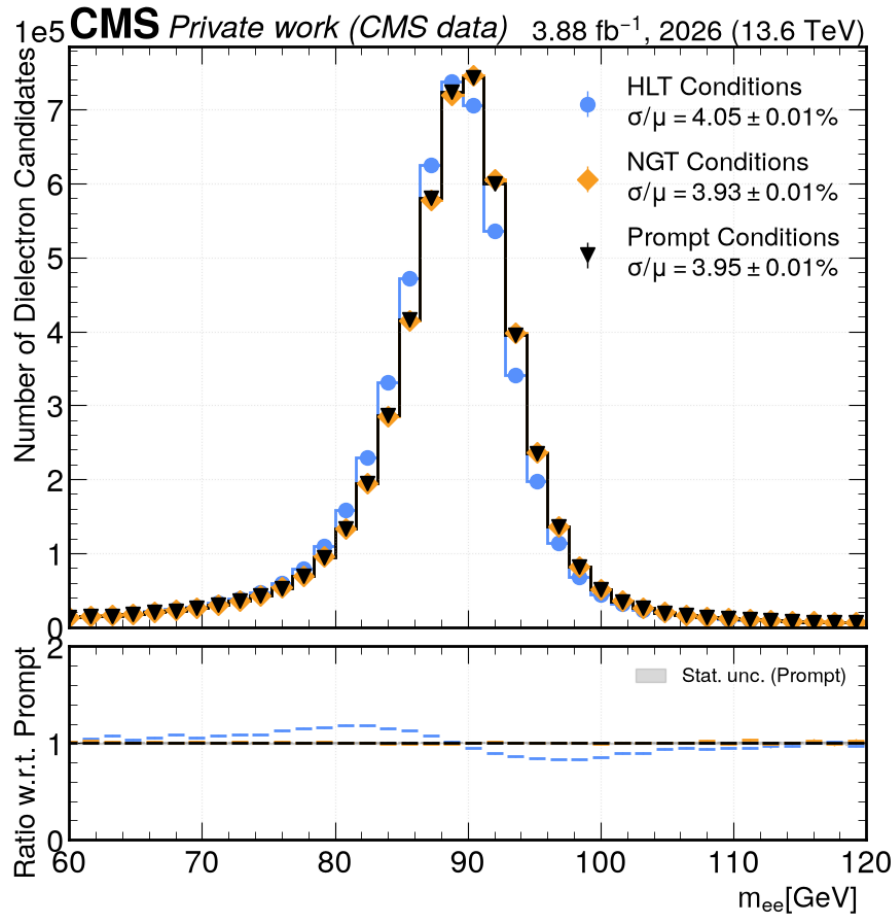


Figure 6: Invariant mass distribution of the dielectron system for $Z \rightarrow e^+e^-$ candidates, reconstructed with the standard HLT conditions (blue circles), the NGT conditions (orange triangles) and the prompt offline conditions (black downward triangles).

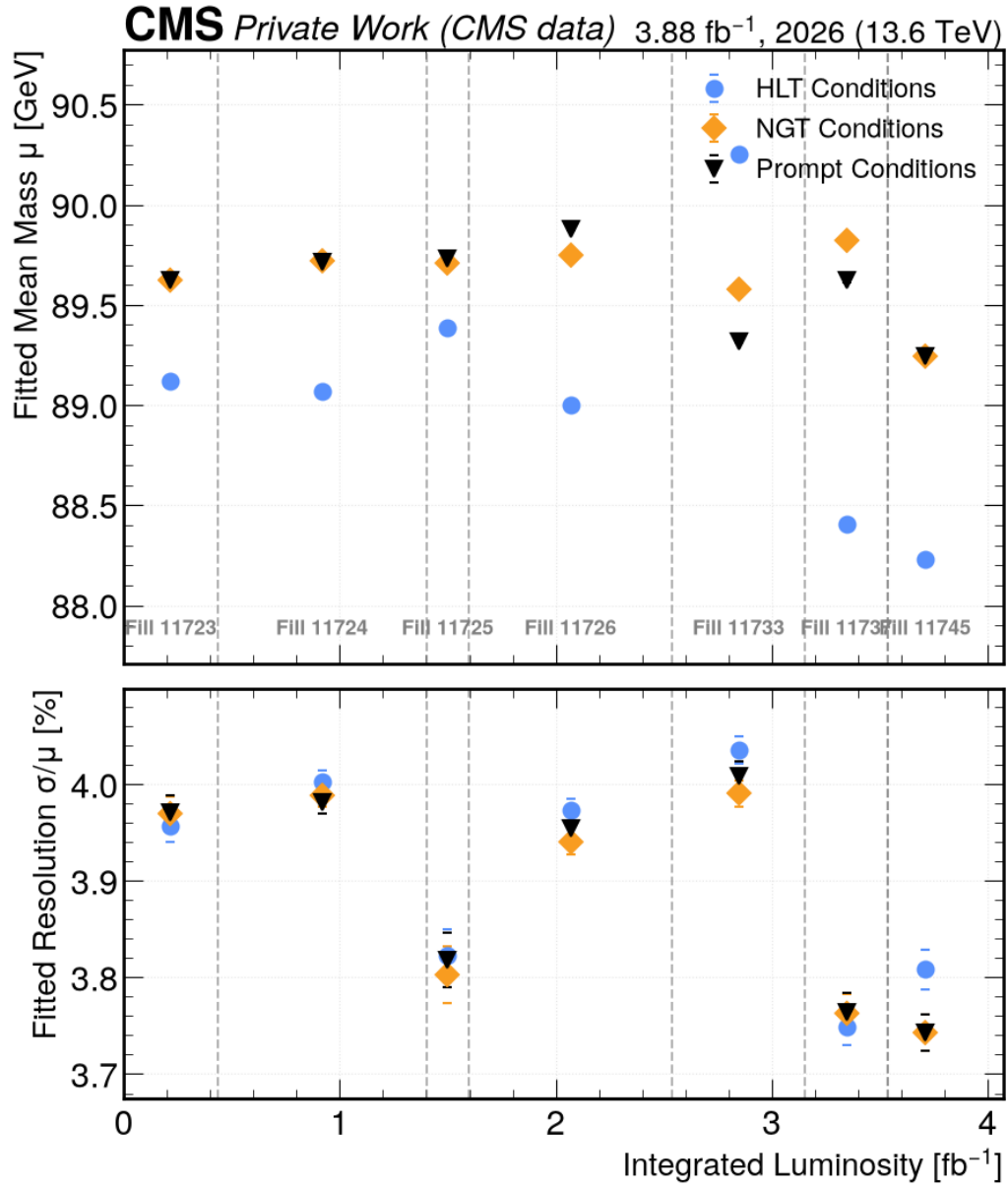


Figure 7: Per-fill stability of the fitted mean mass μ (top) and the relative mass resolution σ/μ (bottom) for $Z \rightarrow e^+e^-$ candidates, as a function of integrated luminosity. Markers as in Figure 6.



son of DQM histograms. Applied to 14 runs of the Run2026D EGamma datasets, the pipeline shows that the $Z \rightarrow e^+e^-$ invariant mass reconstructed with the NGT conditions closely follows the offline prompt reference in both peak position and resolution, where the standard HLT conditions show a clear shift towards lower masses, and that this agreement is stable across fills. This indicates that continuously derived calibrations can bring HLT reconstruction quality close to that of the prompt reconstruction, supporting the feasibility of the continuous calibration approach for Phase-2. A more complete assessment – in particular of the ECAL endcap response, where the largest effect of the more frequent transparency and pedestal updates is expected – is ongoing, and its results will further sharpen this picture.





6 REFERENCES

- [1] Virginia Azzolini et al. “The Data Quality Monitoring Software for the CMS experiment at the LHC: past, present and future”. en. In: *EPJ Web of Conferences* 214 (2019). Ed. by A. Forti et al., p. 02003. ISSN: 2100-014X. DOI: [10.1051/epjconf/201921402003](https://doi.org/10.1051/epjconf/201921402003). URL: <https://www.epj-conferences.org/10.1051/epjconf/201921402003> (visited on 08/22/2026).
- [2] Josh Bendavid. “From Raw Data to Physics Results Part 1”. en. In: *The Large Hadron Collider* (). URL: <https://indico.cern.ch/event/1508891/contributions/6520049/attachments/3096654/5486078/CERNSSLDDataFlow-Jul2025Part1.pdf?page=26>.
- [3] Andrea Bocci et al. *Optimal HLT Calibrations — NGT Task 3.4 2025 Report*. Tech. rep. May 2025.
- [4] CMS Collaboration. *Physics Performance assessment of the Run 3 Optimal Calibrations Next Generation Triggers Demonstrator*. May 2026. URL: https://cds.cern.ch/record/2961610/files/DP2026_028.pdf.
- [5] Federico De Guio. “CMS Alignment and Calibration workflows: lesson learned and future plans”. en. In: *Nuclear and Particle Physics Proceedings* 273-275 (Apr. 2016), pp. 923–928. ISSN: 24056014. DOI: [10.1016/j.nuclphysbps.2015.09.143](https://doi.org/10.1016/j.nuclphysbps.2015.09.143). URL: <https://linkinghub.elsevier.com/retrieve/pii/S240560141500632X> (visited on 08/21/2026).
- [6] “Enriching the physics program of the CMS experiment via data scouting and data parking”. In: *Physics Reports*. CMS physics results from the first decade of LHC data 1115 (Apr. 2025), pp. 678–772. ISSN: 0370-1573. DOI: [10.1016/j.physrep.2024.09.006](https://doi.org/10.1016/j.physrep.2024.09.006). URL: <https://www.sciencedirect.com/science/article/pii/S0370157324003247> (visited on 08/31/2026).
- [7] Carson Glines. *Calibration Profiling & Database Solutions for Optimal NGT-CMS-HLT Calibrations*. CERN Openlab Report. CERN, Aug. 2025.
- [8] S. Di Guida et al. “The CMS Condition Database System”. en. In: *Journal of Physics: Conference Series* 664.4 (Dec. 2015), p. 042024. ISSN: 1742-6588, 1742-6596. DOI: [10.1088/1742-6596/664/4/042024](https://doi.org/10.1088/1742-6596/664/4/042024). URL: <https://iopscience.iop.org/article/10.1088/1742-6596/664/4/042024> (visited on 08/21/2026).
- [9] *LHC Programme Coordination Home Page*. URL: <https://lpc.web.cern.ch/doc/fillingschemes.html> (visited on 08/23/2026).
- [10] M. Musich and M. Grünwald. *Integration and Testing of High-Level Triggers (cmssw, ConfDB)*. CERN, Feb. 2025. URL: https://indico.cern.ch/event/1478254/contributions/6226715/attachments/3022234/5333273/mm_HLT_Tutorial_2025_STORM.pdf.
- [11] *SWGuideHltGetConfiguration < CMSPublic < TWiki*. URL: <https://twiki.cern.ch/twiki/bin/view/CMSPublic/SWGuideHltGetConfiguration> (visited on 08/23/2026).
- [12] *T311 – nextgentriggers.web.cern.ch*. en-GB. URL: <https://nextgentriggers.web.cern.ch/t311/> (visited on 08/25/2026).
- [13] Douglas Thain, Todd Tannenbaum, and Miron Livny. “Distributed computing in practice: the Condor experience”. en. In: *Concurrency and Computation: Practice and Experience* 17.2-4 (Feb. 2005), pp. 323–356. ISSN: 1532-0626, 1532-0634. DOI: [10.1002/cpe.938](https://doi.org/10.1002/cpe.938). URL: <https://onlinelibrary.wiley.com/doi/10.1002/cpe.938> (visited on 08/24/2026).



- [14] Elena Vernazza. “Revolutionizing the CMS High-Level Trigger system at the HL-LHC with Next-Generation Triggers”. en. In: (2026). URL: <https://indico.global/event/484/contributions/143425/attachments/67397/130622/EV%20-%20Revolutionizing%20the%20CMS%20High-Level%20Trigger%20System%20at%20HL-LHC%20with%20Next%20Generation%20Triggers.pdf>.
- [15] *WorkBookConfigFileIntro* < *CMSPublic* < *TWiki*. URL: <https://twiki.cern.ch/twiki/bin/view/CMSPublic/WorkBookConfigFileIntro> (visited on 08/23/2026).

